

Runnable：把写逻辑变成组装 chain 已付费

原创 神说要有光 神光的幸福生活 2026年2月19日 21:55 山东

LangChain 很多 api 都实现了 Runnable 接口，比如 PromptTemplate、OutputParser、ChatOpenAI 等。

不信你可以看一下：

神光的编程秘籍

而且 Runnable 相关的 api 也有很多：

```
11
12 Runnable:
    [Runnable @langchain/core/runnables]
    [RunnableAssign @langchain/core/runnables]
    [RunnableBinding @langchain/core/runnables]
    [RunnableBranch @langchain/core/runnables]
    [RunnableEach @langchain/core/runnables]
    [RunnableLambda @langchain/core/runnables]
    [RunnableMap @langchain/core/runnables]
    [RunnableParallel @langchain/core/runnables]
    [RunnablePassthrough @langchain/core/runnables]
    [RunnablePick @langchain/core/runnables]
    [RunnableRetry @langchain/core/runnables]
    [RunnableSequence @langchain/core/runnables]
```

那 Runnable 都是干什么的呢？

它可以让我们声明式的写代码，从写逻辑变成组装 chain。

我们试一下：

```
mkdir runnable-test  
cd runnable-test  
npm init -y
```

进入项目安装依赖：

```
pnpm install dotenv @langchain/core @langchain/openai zod
```

创建.env 文件：

```
# OpenAI API 配置  
OPENAI_API_KEY=sk-xxx  
OPENAI_BASE_URL=https://dashscope.aliyuncs.com/compatible-mode/v1  
MODEL_NAME=qwen-plus  
EMBEDDINGS_MODEL_NAME=text-embedding-v3
```

之前我们这么写

src/before.mjs

```

import 'dotenv/config';
import { StructuredOutputParser } from"@langchain/core/output_parsers";
import { PromptTemplate } from"@langchain/core/prompts";
import { ChatOpenAI } from"@langchain/openai";
import { z } from"zod";

const model = new ChatOpenAI({
  modelName: process.env.MODEL_NAME,
  apiKey: process.env.OPENAI_API_KEY,
  temperature: 0,
  configuration: {
    baseUrl: process.env.OPENAI_BASE_URL,
  },
});

// 定义输出结构 schema
const schema = z.object({
  translation: z.string().describe("翻译后的英文文本"),
  keywords: z.array(z.string()).length(3).describe("3个关键词")
});

const outputParser = StructuredOutputParser.fromZodSchema(schema);

const promptTemplate = PromptTemplate.fromTemplate(
  '将以下文本翻译成英文，然后总结为3个关键词。\\n\\n文本: {text}\\n\\n{format_instructions}'
);

const input = {
  text: 'LangChain 是一个强大的 AI 应用开发框架',
  format_instructions: outputParser.getFormatInstructions()
};

// 步骤 1: 格式化 prompt
const formattedPrompt = await promptTemplate.format(input);
// 步骤 2: 调用模型
const response = await model.invoke(formattedPrompt);
// 步骤 3: 解析输出
const result = await outputParser.invoke(response);
console.log('✅ 最终结果:');
console.log(result);

```

用 PromptTemplate 管理 prompt，调用 format 传入占位符的值。

调用 ChatOpenAI 的大模型，通过 invoke 方法

用 StructuredOutputParser 做结构化解析，调用 invoke

有了 Runnable 可以这么写：

src/runnable.mjs

```
import 'dotenv/config';
import { StructuredOutputParser } from "@langchain/core/output_parsers";
import { PromptTemplate } from "@langchain/core/prompts";
import { ChatOpenAI } from "@langchain/openai";
import { RunnableSequence } from "@langchain/core/runnables";
import { z } from "zod";

const model = new ChatOpenAI({
  modelName: process.env.MODEL_NAME,
  apiKey: process.env.OPENAI_API_KEY,
  temperature: 0,
  configuration: {
    baseUrl: process.env.OPENAI_BASE_URL,
  },
});

// 定义输出结构 schema
const schema = z.object({
  translation: z.string().describe("翻译后的英文文本"),
  keywords: z.array(z.string()).length(3).describe("3个关键词")
});

const outputParser = StructuredOutputParser.fromZodSchema(schema);

const promptTemplate = PromptTemplate.fromTemplate(
  '将以下文本翻译成英文，然后总结为3个关键词。\\n\\n文本: {text}\\n\\n{format_instructions}'
);

const chain = RunnableSequence.from([
  promptTemplate,
  model,
  outputParser
]);

const input = {
  text: 'LangChain 是一个强大的 AI 应用开发框架',
  format_instructions: outputParser.getFormatInstructions()
};
```

```
const result = await chain.invoke(input);

console.log('✅ 最终结果:');
console.log(result);
```

用 `RunnableSequence` 声明这三个顺序执行，然后直接执行这条 `chain` 就好了。

除了 `RunnableSequence` 声明，还可以直接 `pipe`：

跑下试试：

pipe 的源码里，也是返回 `RunnableSequence`，这俩本质一样：

神光的编程秘籍

明显用了 Runnable 之后，代码简洁了很多。

实现了 Runnable api 后，就可以声明式的组合执行的 chain，然后统一执行。

这种声明式的写法叫做 LCEL (Lang Chain Expression Language, LangChain 表达式语言)

LCEL 就是实现了 Runnbale 接口的一些 api 组合成 chain, 然后统一执行。

Runnable 都有 invoke、stream、batch 方法。

这样你调用 `invoke`，就会依次调用这个链条上每个组件的 `invoke`

`batch` 是批量，也就是并发进行多个单独的 `invoke`

调用 `stream` 就是调用这个链条上每个组件的 `stream`，不断返回数据。

串联起来的 Runnable 的 chain 就自然可以支持同步调用、批量调用、流式返回。

那我们就只需要考虑怎么组装这个 chain 了。

这就是用 LCEL 的方式写代码的好处，你不需要依次调用每个组件，只需要声明这个 chain，然后统一调用。

前面用了 RunnableSequence，它是顺序执行，我们再来用一下其余的 Runnable api：

首先是 RunnableLambda，这个是把普通函数封装成 Runnable 对象

src/runnables/RunnableLambda.mjs

```
import 'dotenv/config';
import { RunnableLambda, RunnableSequence } from "@langchain/core/runnables";

const addOne = RunnableLambda.from((input) => {
  console.log(`输入: ${input}`);
  return input + 1;
});

const multiplyTwo = RunnableLambda.from((input) => {
  console.log(`输入: ${input}`);
  return input * 2;
});

const chain = RunnableSequence.from([
  addOne,
  multiplyTwo,
  addOne
]);

const result = await chain.invoke(5);
console.log(result);
```

我们把两个函数通过 RunnableLambda 封装成了 Runnable 对象

然后通过 RunnableSequence 来顺序调用。

这样，普通函数就可以在这个 chain 里调用了。

然后是 RunnableMap，它可以并行执行多个 Runnable：

src/runnables/RunnableMap.mjs

```
import 'dotenv/config';
import { RunnableMap, RunnableLambda } from "@langchain/core/runnables";
import { PromptTemplate } from "@langchain/core/prompts";

const addOne = RunnableLambda.from((input) => input.num + 1);
const multiplyTwo = RunnableLambda.from((input) => input.num * 2);
const square = RunnableLambda.from((input) => input.num * input.num);

const greetTemplate = PromptTemplate.fromTemplate("你好, {name}! ");
const weatherTemplate = PromptTemplate.fromTemplate("今天天气{weather}。");

// 创建 RunnableMap, 并行执行多个 runnable
const runnableMap = RunnableMap.from({
  // 数学运算
  add: addOne,
  multiply: multiplyTwo,
  square: square,

  // prompt 格式化
  greeting: greetTemplate,
  weather: weatherTemplate,
});

// 测试输入
const input = {
  name: "神光",
```

```

    weather: "多云",
    num: 5,
  };

  // 执行 RunnableMap
  const result = await runnableMap.invoke(input);
  console.log(result);

```

这里我们输入的 input 会并行经过 5 个 Runnable 处理，结果放到对象的对应属性上

接下来是 RunnableBranch，它就是 if else 逻辑

src/runnable /RunnableBranch.mjs

```

import 'dotenv/config';
import { RunnableBranch, RunnableLambda } from "@langchain/core/runnables";

// 创建条件判断函数
const isPositive = RunnableLambda.from((input) => input > 0);
const isNegative = RunnableLambda.from((input) => input < 0);
const isEven = RunnableLambda.from((input) => input % 2 === 0);

// 创建分支处理函数
const handlePositive = RunnableLambda.from((input) => `正数: ${input} + 10 = ${input + 10}`);
const handleNegative = RunnableLambda.from((input) => `负数: ${input} - 10 = ${input - 10}`);
const handleEven = RunnableLambda.from((input) => `偶数: ${input} * 2 = ${input * 2}`);
const handleDefault = RunnableLambda.from((input) => `默认: ${input}`);

// 创建 RunnableBranch

```

```

const branch = RunnableBranch.from([
  [isPositive, handlePositive],
  [isNegative, handleNegative],
  [isEven, handleEven],
  handleDefault
]);

// 测试不同的输入
const testCases = [5, -3, 4, 0];

for (const testCase of testCases) {
  const result = await branch.invoke(testCase);
  console.log(`输入: ${testCase} => ${result}`);
}

```

这里分别对正数、负数、偶数等做不同处理，也就是 if else 的逻辑。

跑一下：

然后是 RouterRunnable，它相当于 switch case

src/runnables/RouterRunnable.mjs

```

import 'dotenv/config';
import { RouterRunnable, RunnableLambda } from "@langchain/core/runnables";

// 创建两个简单的 RunnableLambda
const toUpperCase = RunnableLambda.from((text) => text.toUpperCase());
const reverseText = RunnableLambda.from((text) => text.split("").reverse().join(""));

// 创建 RouterRunnable, 根据 key 选择要调用的 runnable
const router = new RouterRunnable({
  runnables: {
    toUpperCase,
    reverseText,
  },
});

// 测试: 调用 reverseText
const result1 = await router.invoke({ key: "reverseText", input: "Hello World" });

```

```

console.log('reverseText 结果:', result1);

// 测试: 调用 toUpperCase
const result2 = await router.invoke({ key: "toUpperCase", input: "Hello World" });
console.log('toUpperCase 结果:', result2);

```

根据 key 匹配对应的 chain 来执行。

然后是 RunnablePassthrough，它是传入的最初的值：

```

import 'dotenv/config';
import { RunnablePassthrough, RunnableLambda, RunnableSequence, RunnableMap } from "@langchain/core/runtimes";

const chain = RunnableSequence.from([
  RunnableLambda.from((input) => ({ concept: input })),
  RunnableMap.from({
    original: new RunnablePassthrough(),
    processed: RunnableLambda.from((obj) => ({
      concept: input,
      upper: obj.concept.toUpperCase(),
      length: obj.concept.length,
    })),
  })
]);

const input = "神说要有光";
const result = await chain.invoke(input);
console.log(result);

```

我们先用 RunnableLambda 对输入做了转换，然后用 RunnableMap 并行处理。

original 用 RunnablePassthrough 拿到原始值

processed 部分用 RunnableLambda 处理。

可以看到 original 部分就是通过 RunnablePassthrough 拿到了原始值。

这段代码还可以简化：

只保留函数、对象即可，LangChain 会把函数转为 RunnableLambda，把对象转为 RunnableMap

如果是想保留原始属性，只是扩展一些属性，用 RunnablePassthrough.assign

```
const chain = RunnableSequence.from([
  (input) => ({ concept: input }),
  RunnablePassthrough.assign({
    original: new RunnablePassthrough(),
    processed: (obj) => ({
      concept: input,
      upper: obj.concept.toUpperCase(),
      length: obj.concept.length,
    })
  })
]);
```

现在之前的属性也保留着，只是合并了新的属性，就像 `Object.assign` 一样。

接下来是 `RunnableEach`，这个显然就是循环：

`src/runnables/RunnableEach.mjs`

```
import 'dotenv/config';
import { RunnableEach, RunnableLambda, RunnableSequence } from "@langchain/core/runnables"

const toUpperCase = RunnableLambda.from((input) => input.toUpperCase());
const addGreeting = RunnableLambda.from((input) => `你好, ${input}! `);

const processItem = RunnableSequence.from([
  toUpperCase,
  addGreeting,
]);

// 使用 RunnableEach 对数组中的每个元素应用这个链
const chain = new RunnableEach({
  bound: processItem,
});

const input = ["alice", "bob", "carol"];
const result = await chain.invoke(input);

console.log('✅ RunnableEach - 数组元素处理:');
console.log('输入:', input);
console.log('输出:', result);
```

对输入的数组的每个元素应用这个 chain

再就是 RunnablePick，这个就是从对象里取一些属性：

src/runnables/RunnablePick.mjs

```
import 'dotenv/config';
import { RunnablePick, RunnableSequence } from"@langchain/core/runnables";

const inputData = {
  name: "神光",
  age: 30,
  city: "北京",
  country: "中国",
  email: "shenguang@example.com",
  phone: "+86-13800138000",
};

const chain = RunnableSequence.from([
  (input) => ({
    ...input,
    fullInfo: `${input.name}, ${input.age}岁, 来自${input.city}`,
  }),
  new RunnablePick(["name", "fullInfo"]),
]);

const result = await chain.invoke(inputData);
console.log(result);
```

最后是 RunnableWithMessageHistory，它是给 chain 加上 memory 的功能

src/runnables/RunnableWithMessageHistory.mjs

```
import 'dotenv/config';
import { RunnableWithMessageHistory } from"@langchain/core/runnables";
import { InMemoryChatMessageHistory } from"@langchain/core/chat_history";
```

```

import { ChatOpenAI } from"@langchain/openai";
import { ChatPromptTemplate, MessagesPlaceholder } from"@langchain/core/prompts";
import { StringOutputParser } from"@langchain/core/output_parsers";

const model = new ChatOpenAI({
  modelName: process.env.MODEL_NAME,
  apiKey: process.env.OPENAI_API_KEY,
  temperature: 0.3,
  configuration: {
    baseUrl: process.env.OPENAI_BASE_URL,
  },
});

const prompt = ChatPromptTemplate.fromMessages([
  [
    "system",
    "你是一个简洁、有帮助的中文助手，会用 1-2 句话回答用户问题，重点给出明确、有用的信息。",
  ],
  new MessagesPlaceholder("history"),
  ["human", "{question}"],
]);

const simpleChain = prompt.pipe(model).pipe(new StringOutputParser());

const messageHistories = newMap();

const getMessageHistory = (sessionId) => {
  if (!messageHistories.has(sessionId)) {
    messageHistories.set(sessionId, new InMemoryChatMessageHistory());
  }
  return messageHistories.get(sessionId);
};

// 创建带消息历史的链
const chain = new RunnableWithMessageHistory({
  runnable: simpleChain,
  getMessageHistory: (sessionId) => getMessageHistory(sessionId),
  inputMessagesKey: "question",
  historyMessagesKey: "history",
});

// 测试：第一次对话
console.log('--- 第一次对话（提供信息） ---');
const result1 = await chain.invoke(
  {
    question: "我的名字是神光，我来自山东，我喜欢编程、写作、金铲铲。",
  },
  {

```



```

        configurable: {
            sessionId: "user-123",
        },
    },
};

console.log('问题: 我的名字是神光, 我来自山东, 我喜欢编程、写作、金铲铲。');
console.log('回答:', result1);
console.log();

// 测试: 第二次对话
console.log('--- 第二次对话 (询问之前的信息) ---');
const result2 = await chain.invoke(
    {
        question: "我刚才说我来哪里? ",
    },
    {
        configurable: {
            sessionId: "user-123",
        },
    }
);
console.log('问题: 我刚才说我来哪里? ');
console.log('回答:', result2);
console.log();

// 测试: 第三次对话
console.log('--- 第三次对话 (继续询问) ---');
const result3 = await chain.invoke(
    {
        question: "我的爱好是什么? ",
    },
    {
        configurable: {
            sessionId: "user-123",
        },
    }
);
console.log('问题: 我的爱好是什么? ');
console.log('回答:', result3);
console.log();

```

我们用 ChatPromptTemplate 创建 prompt, 其中 history 对话历史用 MessagesPlaceholder 插入。

在 map 里管理每个 sessionId 对应的 ChatMessageHistory

然后创建 RunnableWithMessageHistory 的 chain，告诉它问题、回答都是从哪个字段取

跑一下：

神光的编程秘籍

这样就可以给一段 Chain 加上 memory。

代码上传了课程仓库：<https://github.com/QuarkGluonPlasma/ai-agent-course-code>

总结

LangChain 的很多组件都继承了 Runnable 抽象类，而且也提供了很多 Runnable 的 api

基于 Runnable 的 api 可以很简洁的组装好一条 chain，不用再写很多逻辑

这个叫做 LangChain 表达式语言（LCEL）

我们学了很多 Runnable 的 api：

- RunnableSequence：顺序执行
- RunnableLambda：把函数包装成 Runnable
- RunnableMap：并行执行多个 chain，结果放在对象属性上

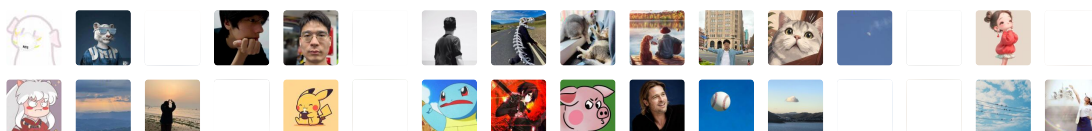
- RunnableBranch: if else 逻辑
- RouterRunnable: switch case 逻辑, 根据 key 决定执行哪个 chain
- RunnableEach: 循环数组每个元素来调用 chain
- RunnablePassthrough: 拿到原始输入
- RunnablePick: 取输入对象的某些属性返回
- RunnableWithMessageHistory: 给 chain 加上 memory

有了这些 Runnable 的 api, 我们可以轻松组装出各种 chain。

并且 Runnable 提供了 invoke、stream、batch 方法, 可以做同步调用、流式返回、批量调用等。

学完 Runnable 的 api 后, 我们再写之前的逻辑, 就都可以用 chain 的方式写了, 这样更简洁。

2774 人付费



转型 Agent 全栈工程师: 企业级知识库项目 · 目录

上一篇

Prompt Template: 组件化管理 prompt

下一篇

实战练习 LCEL 组装 chain

